Module 8: Styling in React – Detailed Notes

Styling is an essential part of building user interfaces in React. React supports multiple styling approaches, ranging from traditional CSS to modern JavaScript-based styling techniques. Choosing the correct styling method improves maintainability, readability, and scalability of React applications.

---

1. CSS Files in React

Concept

CSS files are the most common and traditional method of styling web applications. In React, external CSS files can be imported directly into JavaScript files. The styles defined in these files apply globally to the entire application unless scoped manually.

How It Works in React

- CSS rules are written in .css files.
- These files are imported into React components.
- The className attribute is used instead of the HTML class keyword.

Example

CSS File

```
.heading {
  color: navy;
  font-size: 28px;
  text-align: center;
}
```

React Component

```
import './App.css';

function App() {
  return <h1 className="heading">React Styling</h1>;
}
```

Advantages

- Simple and beginner-friendly
- Separation of concerns between structure and styling
- Easy to reuse styles across components

Disadvantages

- Styles are global by default
- Risk of class name conflicts in large applications

Use Case

Best suited for:

- Small to medium applications
- Common layout and theme styles

Task

Create a CSS file that styles:

- A page heading
- A paragraph
- A submit button

---

2. Inline Styles in React

Concept

Inline styles are applied directly within JSX elements using JavaScript objects. This approach allows dynamic styling based on component logic or state.

Key Characteristics

- Styles are written as JavaScript objects
- Property names use camelCase (backgroundColor)
- Values are strings or numbers

Example

```
function InlineStyle() {
  const textStyle = {
    color: 'green',
    fontSize: '18px',
    marginTop: '10px'
  };

  return <p style={textStyle}>This is inline styled text</p>;
}
```

Advantages

- Dynamic and conditional styling
- No external CSS file required
- Useful for quick custom styles

Limitations

- Cannot use pseudo-classes like :hover
- Not suitable for large or reusable styles
- Reduces readability if overused

Use Case

- Dynamic styling (based on props or state)
- Quick one-time styles

Task

Apply inline styles to:

- Change text color dynamically
- Add padding and margin to an element

---

3. CSS Modules

Concept

CSS Modules provide locally scoped CSS by default. Each CSS class is transformed into a unique name, preventing global namespace pollution.

File Structure

CSS Module files use the naming convention:

ComponentName.module.css

How It Works

- CSS classes are imported as JavaScript objects
- Classes are accessed using object properties

Example

CSS Module

```
.card {
  border: 1px solid #ccc;
  padding: 15px;
  border-radius: 8px;
```

```
}
```

React Component

```
import styles from './Card.module.css';


function Card() {
  return <div className={styles.card}>Card Content</div>;
}
```

Advantages

- Avoids class name conflicts
- Improves code maintainability
- Ideal for component-based styling

Disadvantages

- Slightly more complex syntax
- Less suitable for global styles

Use Case

- Medium to large applications
- Component-level styling

Task

Create a CSS Module to style:

- A reusable card component
- A button with hover effect

---

4. Styled Components (Introduction)

Concept

Styled Components is a popular **CSS-in-JS** library that allows developers to define styles directly inside JavaScript files. Each styled component is a React component with encapsulated styles.

Installation

```
npm install styled-components
```

Basic Example

```
import styled from 'styled-components';
```

```
const Button = styled.button`
  background-color: teal;
  color: white;
  padding: 10px 15px;
  border: none;
`;

function StyledComponentExample() {
  return <Button>Styled Button</Button>;
}
```

Features

- Styles are scoped to components
- Supports dynamic styling using props
- Eliminates the need for separate CSS files

Advantages

- Clean and modular styling
- Easy to maintain in large applications
- Supports theming and conditional styles

Disadvantages

- Additional dependency
- Slight performance overhead

Use Case

- Large-scale React applications
- Applications requiring dynamic and theme-based styling

Task

Create styled components for:

- A heading
- A button that changes color on hover

---

5. Responsive Design Basics in React

Concept

Responsive design ensures that React applications adapt to different screen sizes such as mobile, tablet, and desktop.

Techniques Used

- Media queries
- Flexible layouts
- Relative units

Media Query Example

```
.container {
  padding: 20px;
}

@media (max-width: 768px) {
  .container {
    padding: 10px;
  }
}
```

Flexible Units

- % for relative width
- vw and vh for viewport-based sizing
- rem and em for scalable typography

Responsive Layout Example

```
<div className="container">
  Responsive Content
</div>
```

Advantages

- Better user experience
- Mobile-first design
- Improved accessibility

Task

Create a responsive component where:

- Font size decreases on smaller screens

- Layout switches from horizontal to vertical

Module 9: Forms & User Input in React

Forms are an essential part of web applications as they allow users to enter data and interact with the system. In React, form handling is done using **controlled components**, where form data is managed by the component's state.

---

1. Handling Forms in React

Concept

In React, forms are handled differently from traditional HTML forms. Instead of allowing the browser to manage form data, React uses **state** to control form inputs. This approach provides better control, validation, and real-time updates.

Controlled Components

A controlled component is an input element whose value is controlled by React state.

Key Points

- Form data is stored in state
- onChange event updates the state
- onSubmit event handles form submission
- event.preventDefault() prevents page reload

Basic Form Example

```
import { useState } from "react";


function SimpleForm() {
  const [name, setName] = useState("");


  const handleSubmit = (e) => {
    e.preventDefault();
    alert("Submitted Name: " + name);
  };


  return (
    <form onSubmit={handleSubmit}>
```

```
    <label>Name:</label>
    <input
      type="text"
      value={name}
      onChange={(e) => setName(e.target.value)}
    />
    <button type="submit">Submit</button>
  </form>
 );
}
```

Advantages of Controlled Forms

- Better validation control
- Immediate access to user input
- Predictable data flow

Task

✅ Create a form that accepts:

- Username
- Email
- Submit button

---

2. Input Fields in React

Concept

React supports various input fields such as text boxes, password fields, radio buttons, checkboxes, and select dropdowns. Each input field is usually linked to a state variable.

Text Input Field

```
const [email, setEmail] = useState("");


<input
  type="email"
  value={email}
```

```
  onChange={(e) => setEmail(e.target.value)}
/>
```

Password Input Field

```
const [password, setPassword] = useState("");


<input
  type="password"
  value={password}
  onChange={(e) => setPassword(e.target.value)}
/>
```

Checkbox Input

```
const [agree, setAgree] = useState(false);


<input
  type="checkbox"
  checked={agree}
  onChange={(e) => setAgree(e.target.checked)}
/>
```

Radio Button

```
const [gender, setGender] = useState("");


<input
  type="radio"
  value="Male"
  checked={gender === "Male"}
  onChange={(e) => setGender(e.target.value)}
/> Male
```

Select Dropdown

```
const [city, setCity] = useState("");


<select value={city} onChange={(e) => setCity(e.target.value)}>
  <option value="">Select City</option>
  <option value="Delhi">Delhi</option>
  <option value="Mumbai">Mumbai</option>
</select>
```

Task

✅ Create a form that includes:

- Text input
- Password input
- Checkbox
- Dropdown menu

---

3. Form Validation Basics

Concept

Form validation ensures that users enter **correct and complete data** before submitting a form. Validation can be done manually in React using state and conditional logic.

Basic Validation Techniques

- Required field validation
- Minimum length validation
- Email format validation
- Conditional error messages

Example: Required Field Validation

```
import { useState } from "react";


function ValidationForm() {
```

```
  const [email, setEmail] = useState("");
  const [error, setError] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();
    if (email === "") {
      setError("Email is required");
    } else {
      setError("");
      alert("Form submitted successfully");
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
      />
      <p style={{ color: "red" }}>{error}</p>
      <button type="submit">Submit</button>
    </form>
  );
}
```

Password Length Validation Example

```
if (password.length < 6) {
  setError("Password must be at least 6 characters");
}
```

Advantages of Validation

- Prevents incorrect data submission
- Improves user experience
- Ensures data integrity

Task

✅ Implement validation for:

- Empty fields
- Minimum password length
- Valid email format

Mini Assignment (Module 9)

📝 **Create a Registration Form** with:

- Name, Email, Password
- Gender selection
- Checkbox for terms & conditions
- Validation for all fields

Module 10: Handling Events in React

Event handling in React allows components to respond to user interactions such as clicks, key presses, and form submissions. React uses a **synthetic event system**, which provides consistent behavior across different browsers.

---

1. Event Handling in React

Concept

Events in React are similar to DOM events but follow **camelCase naming conventions** and use JSX syntax. Instead of strings, event handlers receive **functions**.

Common React Events

- onClick
- onChange
- onSubmit
- onMouseOver
- onKeyDown

Basic Event Handling Example

```
function ClickExample() {
  const handleClick = () => {
    alert("Button clicked!");
  };

  return <button onClick={handleClick}>Click Me</button>;
}
```

Key Differences from HTML

HTML	React
onclick="myFunc()"	onClick={myFunc}
Uses strings	Uses functions

Advantages

- Cleaner and more readable code
- Better control over application behavior

Task

✅ Create a button that displays a message when clicked.

---

2. Binding Event Handlers

Concept

In React class components, event handlers need to be **bound** to the correct this context. Without binding, this will be undefined inside the handler.

⚠ Binding is mainly required in **class components**.
Functional components using hooks do **not** require binding.

Binding Using Constructor

```
class BindExample extends React.Component {
  constructor(props) {
    super(props);
    this.handleClick = this.handleClick.bind(this);
```

```
  }

  handleClick() {
    alert("Button clicked");
  }

  render() {
    return <button onClick={this.handleClick}>Click</button>;
  }
}
```

Binding Using Arrow Functions

```
class ArrowBind extends React.Component {
  handleClick = () => {
    alert("Button clicked");
  };

  render() {
    return <button onClick={this.handleClick}>Click</button>;
  }
}
```

Why Binding Is Needed

- Ensures correct reference to this
- Prevents runtime errors
- Allows access to state and props

Task

✅ Create a class component with a button and bind the click handler correctly.

---

3. Arrow Functions vs. Regular Functions

Regular Function

```
function regularFunction() {
  console.log(this);
}
```

- this depends on how the function is called
- Requires manual binding in class components

Arrow Function

```
const arrowFunction = () => {
  console.log(this);
};
```

- this is lexically bound
- Automatically refers to the surrounding scope
- No need for binding

Comparison Table

Feature	Regular Function	Arrow Function
this binding	Dynamic	Lexical
Needs binding	Yes (class components)	No
Syntax	Verbose	Short
Performance	Slightly better	Slight overhead

Example in Event Handling

```
<button onClick={() => alert("Clicked!")}>Click</button>
```

Best Practice

- Use **arrow functions** for event handlers
- Avoid inline arrow functions in large lists for performance

---

Task

✅ Rewrite a regular event handler function using an arrow function.

---

4. Handling Submit Events

Concept

Submit events occur when a form is submitted. In React, submit handling is done using the onSubmit event.

Important Point

- Use event.preventDefault() to stop page reload

Submit Event Example

```
function SubmitForm() {
  const handleSubmit = (e) => {
    e.preventDefault();
    alert("Form Submitted");
  };

  return (
    <form onSubmit={handleSubmit}>
      <input type="text" placeholder="Enter name" />
      <button type="submit">Submit</button>
    </form>
  );
}
```

Passing Data on Submit

```
const handleSubmit = (e) => {
  e.preventDefault();
  console.log("Form data submitted");
};
```

Use Cases

- Login forms
- Registration forms
- Search forms

Task

✅ Create a form that:

- Accepts user input
- Displays a message on submit
- Prevents page refresh

Mini Assignment (Module 10)

📝 **Create a Feedback Form**:

- Handle button click events
- Handle form submit events
- Use arrow functions
- Display submitted data on the screen

Mini Real-World Project (Module 10)

📝 **Project Title:** User Interaction Dashboard

Requirements

- Login form (submit event)
- Add-to-cart button (click event)
- Search bar (keyboard event)
- Theme toggle (arrow function)
- Feedback form (multiple events)